

MADAPORC

Dossier de démarrage du code

Spring Boot MVC — Controllers, Services, Repositories, DTO, vues, règles métier et base de données

Informations générales

Élément	Détail
Projet	MADAPORC
Type	Application web de gestion d'élevage porcin
Cœur du projet	Suivi reproductif par groupes, traçabilité des lots, alertes de mise bas et aide à la décision reproductive
Architecture cible	Java / Spring Boot MVC / JPA / Thymeleaf ou JSP
Style de code	@Controller + Model + @ModelAttribute + @RequestParam + @PathVariable + Service + Repository
Version	Document révisé après recentrage métier
Principe important	Le système ne suit pas les porcs individuellement. Il travaille par lots et par groupes reproductifs.

1. Objectif du document

Ce document transforme le cahier des charges révisé de MADAPORC en plan de codage concret pour Spring Boot MVC.

Il indique les éléments à créer :

- Controllers ;
- Services ;
- Repositories ;
- DTO ;
- vues ;
- validations ;
- tables ;
- colonnes principales ;
- règles métier.

Le nouveau cœur du projet est le suivant :

MADAPORC ne se limite pas à gérer des lots de porcs. Le système suit les groupes reproductifs, calcule les dates prévisionnelles de mise bas, génère des alertes, permet la création de lots de naissance et fournit une aide à la décision reproductive au niveau des lots femelles.

2. Conventions de développement

Élément	Détail
Package racine conseillé	<code>com.madaporc</code>
Controllers	<code>com.madaporc.controller</code>
Services	<code>com.madaporc.service</code>
Repositories	<code>com.madaporc.repository</code>
Entities JPA	<code>com.madaporc.model</code>
DTO	<code>com.madaporc.dto</code>
Vues	<code>src/main/resources/templates</code> ou <code>WEB-INF/views</code>
Style Controller	Retourne une vue <code>String</code>
Formulaires	Reçus avec <code>@ModelAttribute</code>
Paramètres	Reçus avec <code>@RequestParam</code> ou <code>@PathVariable</code>
Transactions	<code>@Transactional</code> sur les méthodes qui modifient plusieurs tables
Gestion erreurs	Les services retournent <code>String erreur</code> ou <code>null</code>

3. Exemple de style à respecter

```
@Controller
public class GroupeReproductionController {

    private final GroupeReproductionService groupeReproductionService;

    public GroupeReproductionController(GroupeReproductionService
groupeReproductionService) {
        this.groupeReproductionService = groupeReproductionService;
    }

    @GetMapping("/reproduction/groupe")
    public String listGroupes(Model model) {
        model.addAttribute("groupes", groupeReproductionService.findAll());
    }
}
```

```

        return "reproduction/groupe";
    }

    @GetMapping("/reproduction/groupe/form")
    public String showForm(@RequestParam(required = false) Long id, Model
model) {
        groupeReproductionService.prepareFormModel(model, id);
        return "reproduction/formGroupe";
    }

    @PostMapping("/reproduction/groupe/save")
    public String save(@ModelAttribute GroupeReproductionDTO dto,
        Model model,
        HttpSession session) {

        Long utilisateurId = (Long) session.getAttribute("userId");

        String erreur = dto.getId() == null
            ? groupeReproductionService.creer(dto, utilisateurId)
            : groupeReproductionService.modifier(dto.getId(), dto);

        model.addAttribute("message",
            erreur == null ? "Groupe de reproduction enregistré avec
succès." : erreur);

        groupeReproductionService.prepareFormModel(model, dto.getId());

        return "reproduction/formGroupe";
    }
}

```

4. Règles métier générales

4.1 Gestion par lots

Un lot représente un groupe homogène de porcs.

Règles :

- un lot est soit mâle, soit femelle ;
- un lot ne peut pas être mixte ;
- un lot possède un effectif initial et un effectif actuel ;
- les mouvements expliquent les variations de l'effectif actuel ;
- un lot archivé ne doit plus apparaître dans les lots actifs.

4.2 Gestion par groupes reproductifs

MADAPORC ne suit pas les porcs individuellement.

Lorsqu'une partie d'un lot femelle entre en reproduction, le système crée un groupe reproductif.

Exemple :

```
LOT-F-001
Total femelles : 4

L'utilisateur saillit seulement 2 femelles.

Le système crée :
GR-001
Femelles concernées : 2
Lot mâle utilisé : LOT-M-001
Mâles utilisés : 1
```

Le lot femelle n'est donc pas entièrement en gestation.

Le système affiche :

```
LOT-F-001
Total femelles : 4
Femelles en cycle : 2
Femelles disponibles : 2
```

4.3 Statuts reproductifs au niveau du lot

Le système répartit les femelles d'un lot selon leur état reproductif.

Statuts utilisés :

- prêtes mais jamais saillies ;
- déjà reproductrices et encore aptes ;
- en cycle de reproduction ;
- à surveiller ;
- à retirer de la reproduction.

Exemple :

```
LOT-F-002
Total femelles : 25

Prêtes mais jamais saillies : 12
Déjà reproductrices et encore aptes : 8
En cycle : 0
À surveiller : 3
À retirer de la reproduction : 2
```

4.4 Fertilité observée

Le taux de fertilité ne se calcule pas sur tout le lot.

Il se calcule uniquement sur les femelles déjà saillies.

Formule :

Taux de fertilité observé =
 $\text{nombre de femelles gestantes} / \text{nombre de femelles saillies} \times 100$

Exemple :

Femelles saillies : 10
Femelles gestantes : 8

Taux de fertilité observé = 80 %

4.5 Aptitude reproductive globale

L'aptitude reproductive répond à la question :

Combien de femelles du lot peuvent encore être utilisées pour la reproduction ?

Formule :

Taux d'aptitude globale =
 $(\text{prêtes} + \text{déjà aptes} + \text{à surveiller}) / \text{total femelles} \times 100$

4.6 Femelles recommandées sans risque

Formule :

Taux recommandé =
 $(\text{prêtes} + \text{déjà aptes}) / \text{total femelles} \times 100$

4.7 Alerte de mise bas

Lorsqu'un groupe reproductif est proche de la date prévue de mise bas, le système génère une alerte.

Formule :

```
date_prevue_mise_bas = date_saillie + duree_gestation_jours
```

Par défaut :

```
duree_gestation_jours = 114  
alerte = 5 jours avant la date prévue
```

5. Liste globale des controllers et routes

AuthController

- /login
- /logout

DashboardController

- /dashboard

LotPorcController

- /lots
- /lots/form
- /lots/save
- /lots/{id}
- /lots/archive/{id}

MouvementLotController

- /lots/{id}/mouvements
- /lots/mouvements/save

PeseeLotController

- /lots/{id}/pesees
- /lots/pesees/save

RepartitionReproductiveController

- /lots/{id}/repartition-reproductive
- /lots/repartition-reproductive/save

GroupeReproductionController

- /reproduction/groupe
- /reproduction/groupe/form

- /reproduction/groupe/save
- /reproduction/groupe/{id}
- /reproduction/groupe/{id}/confirmer-mise-bas
- /reproduction/groupe/{id}/cloturer

AlerteReproductionController

- /reproduction/alertes
- /reproduction/alertes/{id}/lire
- /reproduction/alertes/{id}/traiter

AnalyseReproductionController

- /reproduction/analyse/lots/{lotId}
- /reproduction/analyse/generer/{lotId}

ClientController

- /clients
- /clients/form
- /clients/save
- /clients/{id}

VenteController

- /ventes
- /ventes/form
- /ventes/save
- /ventes/valider/{id}
- /ventes/annuler/{id}
- /ventes/recu/pdf/{id}

DepenseController

- /depenses
- /depenses/form
- /depenses/save

IngredientController

- /ingredients
- /ingredients/form
- /ingredients/save

MouvementStockController

- /stocks/mouvements
- /stocks/mouvements/form
- /stocks/mouvements/save

VaccinController

- /vaccins
- /vaccins/form
- /vaccins/save

VaccinationController

- /vaccinations
- /vaccinations/form
- /vaccinations/save

SuiviSanitaireController

- /sante/suivis
- /sante/suivis/form
- /sante/suivis/save

EmployeController

- /employes
- /employes/form
- /employes/save

PresenceController

- /presences
- /presences/save
- /presences/pointer

RapportController

- /rapports
- /rapports/sanitaire/pdf
- /rapports/commercial/pdf
- /rapports/financier/pdf
- /rapports/reproduction/pdf
- /rapports/export/excel

ImportExportController

- /imports
- /imports/form
- /imports/excel
- /exports/excel
- /exports/pdf
- /imports-exports/historique

ProfilController

- /profil
- /profil/save
- /profil/password

UtilisateurController

- /utilisateurs
 - /utilisateurs/form
 - /utilisateurs/save
 - /utilisateurs/desactiver/{id}
-

6. Controllers supprimés

Les controllers suivants ne font plus partie du périmètre final :

- ReproducteurController
- EvenementReproductionController
- PaiementFactureController
- MelangeController
- DistributionAlimentController
- SalaireController

Justification :

- le projet ne suit pas les porcs individuellement ;
 - la reproduction est gérée par groupes ;
 - les paiements et factures avancées sont remplacés par un reçu PDF simple ;
 - les mélanges, distributions et salaires sont reportés.
-

7. Détail des pages, fonctions et paramètres

7.1 Page de connexion

Élément	Détail
Vue Spring MVC	login
Controller	AuthController

Affichage à prévoir

- email ;
- mot de passe ;

- bouton connexion ;
- message d'erreur ;
- redirection vers dashboard.

Fonctions Spring Boot MVC à créer

Élément	Détail
Controller	<code>@GetMapping("/login") String showLogin(Model model)</code>
Controller	<code>@PostMapping("/login") String login(@ModelAttribute LoginDTO dto, Model model, HttpSession session)</code>
Controller	<code>@GetMapping("/logout") String logout(HttpSession session)</code>
Service	<code>String connecter(LoginDTO dto, HttpSession session)</code>
Service	<code>boolean verifierMotDePasse(String brut, String hash)</code>
Service	<code>Optional<Utilisateur> findByEmail(String email)</code>
Repository	<code>UtilisateurRepository.findByEmail(String email)</code>
DTO	<code>LoginDTO(email:String, motDePasse:String)</code>

Tables utilisées

- utilisateurs ;
- roles.

Règles et validations

- le mot de passe est hashé ;
- un utilisateur désactivé ne peut pas se connecter ;
- la session contient `userId`, `roleId`, `nom`.

7.2 Page tableau de bord

Élément	Détail
Vue Spring MVC	<code>dashboard/index</code>
Controller	<code>DashboardController</code>

Affichage à prévoir

- lots actifs ;
- total porcs ;
- lots femelles reproductifs ;
- groupes de reproduction actifs ;
- mises bas proches ;
- alertes reproduction ;
- taux d'aptitude reproductive ;

- taux de fertilité observé ;
- porcelets nés vivants ;
- ventes du mois ;
- dépenses du mois ;
- bénéfice net ;
- alertes stock ;
- vaccinations à venir.

Fonctions Spring Boot MVC à créer

Élément	Détail
Controller	<code>@GetMapping("/dashboard") String dashboard(Model model, HttpSession session)</code>
Service	<code>DashboardDTO getDashboard(LocalDate debut, LocalDate fin)</code>
Service	<code>long compterLotsActifs()</code>
Service	<code>int calculerTotalPorcsActifs()</code>
Service	<code>long compterGroupesReproductionActifs()</code>
Service	<code>List<AlerteReproduction> listerAlertesActives()</code>
Service	<code>BigDecimal calculerTauxFertiliteGlobal()</code>
Service	<code>BigDecimal calculerTauxAptitudeGlobal()</code>
Service	<code>BigDecimal calculerVentesMois(LocalDate mois)</code>
Service	<code>BigDecimal calculerDepensesMois(LocalDate mois)</code>
Service	<code>BigDecimal calculerBeneficeNet(LocalDate debut, LocalDate fin)</code>
Service	<code>List<Ingredient> listerStocksFaibles()</code>
Service	<code>List<Vaccination> listerVaccinationsAVenir(LocalDate dateLimite)</code>

Tables utilisées

- lots_porcs ;
- groupes_reproduction ;
- repartitions_reproductives_lots ;
- analyses_reproduction_lots ;
- alertes_reproduction ;
- suivis_sanitaires ;
- vaccinations ;
- ventes ;
- details_vente ;
- depenses ;
- ingredients.

Règles et validations

- les statistiques sont calculées depuis la base ;

- les lots archivés ne sont pas actifs ;
- le bénéfice net est calculé ainsi : ventes - dépenses ;
- les alertes de mise bas doivent apparaître en priorité.

7.3 Page liste des lots de porcs

Élément	Détail
Vue Spring MVC	lots/listeLots
Controller	LotPorcController

Affichage à prévoir

- code lot ;
- sexe ;
- race ;
- objectif ;
- origine ;
- effectif initial ;
- effectif actuel ;
- statut ;
- recherche ;
- filtres ;
- pagination ;
- actions voir / modifier / archiver.

Fonctions à créer

Élément	Détail
Controller	@GetMapping("/lots") String listLots(...)
Controller	@PostMapping("/lots/archive/{id}") String archiver(@PathVariable Long id, Model model)
Service	List<LotPorc> rechercherLots(String code, Long raceId, String sexe, String objectif, String statut)
Service	Optional<LotPorc> findLot(Long id)
Service	String archiverLot(Long lotId)
Service	boolean verifierLotActif(Long lotId)
Repository	LotPorcRepository.findByCodeLotContainingIgnoreCase(String code)
Repository	LotPorcRepository.findBySexe(String sexe)
Repository	LotPorcRepository.findByObjectif(String objectif)
DTO	LotFiltreDTO(code:String, raceId:Long, sexe:String, objectif:String, statut:String)

Tables utilisées

- lots_porcs ;
- races.

Règles et validations

- chaque lot possède un code unique ;
- un lot ne peut pas être mixte ;
- sexe obligatoire : MALE ou FEMELLE ;
- effectif initial > 0 ;
- effectif actuel >= 0.

7.4 Page ajout / modification d'un lot

Élément	Détail
Vue Spring MVC	lots/formLot
Controller	LotPorcController

Affichage à prévoir

- code lot ;
- race ;
- sexe ;
- objectif ;
- origine ;
- effectif initial ;
- effectif actuel ;
- lot parent si origine naissance ;
- groupe reproduction origine si naissance ;
- description.

DTO

```
LotPorcDTO(  
    Long id,  
    String codeLot,  
    Long raceId,  
    String sexe,  
    String objectif,  
    String origine,  
    Integer effectifInitial,  
    Integer effectifActuel,  
    Long lotParentId,  
    Long groupeReproductionOrigineId,  
    String description  
)
```

Règles et validations

- code lot unique ;
 - sexe obligatoire ;
 - objectif obligatoire ;
 - origine obligatoire ;
 - effectif initial > 0 ;
 - effectif actuel >= 0 ;
 - si origine = `NAISSANCE`, le lot parent ou le groupe de reproduction origine doit être renseigné ;
 - un lot naissance peut être créé après confirmation de mise bas.
-

7.5 Page détail d'un lot

Élément	Détail
Vue Spring MVC	<code>lots/detailLot</code>
Controller	<code>LotPorcController</code>

Affichage à prévoir

- informations générales ;
- sexe ;
- race ;
- objectif ;
- origine ;
- effectif actuel ;
- mouvements ;
- pesées ;
- santé ;
- vaccinations ;
- ventes ;
- si lot femelle : analyse reproductive ;
- si lot femelle : groupes de reproduction actifs ;
- si lot naissance : lot mère et groupe origine.

Onglets

- informations ;
- mouvements ;
- pesées ;
- santé ;
- vaccinations ;
- ventes ;
- reproduction ;
- analyse reproductive.

Fonctions à créer

Élément	Détail
Service	<code>LotDetailDTO getDetailLot(Long lotId)</code>
Service	<code>List<MouvementLotPorc> getMouvements(Long lotId)</code>
Service	<code>List<PeseeLot> getPesees(Long lotId)</code>
Service	<code>List<SuiviSanitaire> getSuivisSanitaires(Long lotId)</code>
Service	<code>List<Vaccination> getVaccinations(Long lotId)</code>
Service	<code>List<GroupeReproduction> getGroupesReproduction(Long lotId)</code>
Service	<code>AnalyseReproductionLotDTO getAnalyseReproductive(Long lotId)</code>
Service	<code>BigDecimal calculerTauxMortalite(Long lotId)</code>

Tables utilisées

- lots_porcs ;
- mouvements_lots_porcs ;
- pesees_lots ;
- suivis_sanitaires ;
- vaccinations ;
- details_vente ;
- groupes_reproduction ;
- analyses_reproduction_lots ;
- repartitions_reproductives_lots.

7.6 Page mouvements des lots

Élément	Détail
Vue Spring MVC	<code>lots/mouvements</code>
Controller	<code>MouvementLotController</code>

Affichage à prévoir

- liste des mouvements ;
- lot concerné ;
- type mouvement ;
- quantité ;
- date ;
- observation.

DTO

```
MouvementLotDTO(  
    Long id,  
    Long lotId,  
    String typeMouvement,  
    Integer quantite,  
    LocalDate dateMouvement,  
    String observation  
)
```

Règles et validations

- quantité > 0 ;
- une entrée augmente l'effectif actuel ;
- une sortie, vente, décès ou transfert sortie diminue l'effectif actuel ;
- la quantité sortie ne peut pas dépasser l'effectif actuel.

7.7 Page suivi du poids des lots

Élément	Détail
Vue Spring MVC	lots/pesees
Controller	PeseeLotController

Affichage à prévoir

- liste des pesées ;
- date pesée ;
- poids moyen ;
- observation ;
- graphique d'évolution.

DTO

```
PeseeLotDTO(  
    Long id,  
    Long lotId,  
    BigDecimal poidsMoyen,  
    LocalDate datePesee,  
    String observation  
)
```

Règles et validations

- la pesée est liée à un lot existant ;
- poids moyen > 0 ;

- date pesée non future.

8. Module cœur du projet : reproduction par groupes

8.1 Page groupes de reproduction

Élément	Détail
Vue Spring MVC	<code>reproduction/groupe</code>
Controller	<code>GroupeReproductionController</code>

Affichage à prévoir

- code groupe ;
- lot femelle ;
- lot mâle utilisé ;
- nombre de femelles concernées ;
- nombre de mâles utilisés ;
- date de saillie ;
- date prévue de mise bas ;
- statut ;
- alerte ;
- actions détail / confirmer mise bas / clôturer.

Fonctions à créer

Élément	Détail
Controller	<code>@GetMapping("/reproduction/groupe") String listGroupes(Model model)</code>
Controller	<code>@GetMapping("/reproduction/groupe/form") String showForm(...)</code>
Controller	<code>@PostMapping("/reproduction/groupe/save") String save(...)</code>
Controller	<code>@GetMapping("/reproduction/groupe/{id}") String detail(...)</code>
Controller	<code>@PostMapping("/reproduction/groupe/{id}/confirmer-mise-bas") String confirmerMiseBas(...)</code>
Service	<code>List<GroupeReproduction> findAll()</code>
Service	<code>String creer(GroupeReproductionDTO dto, Long utilisateurId)</code>
Service	<code>String modifier(Long id, GroupeReproductionDTO dto)</code>
Service	<code>String confirmerMiseBas(Long id, ConfirmationMiseBasDTO dto)</code>

Élément	Détail
Service	<code>String cloturer(Long id)</code>
Service	<code>LocalDate calculerDatePrevueMiseBas(LocalDate dateSaillie, Integer dureeGestation)</code>
Service	<code>String verifierFemellesDisponibles(Long lotFemelleId, Integer nombreFemelles)</code>
Service	<code>void mettreAJourRepartitionApresSaillie(Long lotId, Integer nombreFemelles)</code>
Service	<code>void mettreAJourRepartitionApresMiseBas(Long lotId, GroupeReproduction groupe)</code>
Repository	<code>GroupeReproductionRepository.findAllByOrderByDateSaillieDesc()</code>
Repository	<code>GroupeReproductionRepository.findByLotFemelleId(Long lotId)</code>
Repository	<code>LotPorcRepository.findBySexeAndObjectif(String sexe, String objectif)</code>

DTO

```
GroupeReproductionDTO(
    Long id,
    String codeGroupe,
    Long lotFemelleId,
    Long lotMaleId,
    Integer nombreFemellesConcernees,
    Integer nombreMalesUtilises,
    LocalDate dateSaillie,
    Integer dureeGestationJours,
    String observation
)
```

Tables utilisées

- groupes_reproduction ;
- lots_porcs ;
- repartitions_reproductives_lots ;
- parametres_reproduction_race ;
- alertes_reproduction.

Règles et validations

- le lot femelle doit être de sexe `FEMELLE` ;
- le lot mâle doit être de sexe `MALE` ;
- seuls les lots femelles peuvent démarrer un groupe de reproduction ;
- le nombre de femelles concernées doit être > 0 ;
- le nombre de femelles concernées ne peut pas dépasser les femelles disponibles du lot ;
- la date prévue de mise bas est calculée automatiquement ;

- le lot entier ne devient pas en gestation, seulement le groupe concerné.

8.2 Page détail d'un groupe de reproduction

Élément	Détail
Vue Spring MVC	reproduction/detailGroupe
Controller	GroupeReproductionController

Affichage à prévoir

- lot femelle source ;
- lot mâle utilisé ;
- nombre de femelles concernées ;
- date de saillie ;
- date prévue de mise bas ;
- nombre de jours restants ;
- statut du groupe ;
- barre d'évolution ;
- porcelets nés ;
- porcelets vivants ;
- porcelets morts ;
- bouton confirmer mise bas.

Barre d'évolution

Saillie ——— Gestation ——— Préparation mise bas ——— Mise bas

Règles et validations

- la barre d'évolution dépend de la date de saillie et de la date prévue de mise bas ;
- l'alerte s'affiche lorsque la mise bas est proche ;
- la mise bas doit être confirmée manuellement par l'utilisateur.

8.3 Confirmation de mise bas

Élément	Détail
Vue Spring MVC	reproduction/confirmerMiseBas
Controller	GroupeReproductionController

Affichage à prévoir

- date réelle de mise bas ;
- nombre de porcelets nés ;
- nombre de porcelets vivants ;

- nombre de porcelets morts ;
- observation ;
- proposition de création d'un nouveau lot naissance.

DTO

```
ConfirmationMiseBasDTO(
    Long groupeReproductionId,
    LocalDate dateMiseBasReelle,
    Integer nbFemellesMiseBas,
    Integer nbPorceletsNes,
    Integer nbPorceletsVivants,
    Integer nbPorceletsMorts,
    Boolean creerLotNaissance,
    String codeLotNaissance,
    String sexeLotNaissance,
    String observation
)
```

Règles et validations

- date réelle de mise bas $\geq$ date de saillie ;
- porcelets nés ≥ 0 ;
- porcelets vivants ≥ 0 ;
- porcelets morts ≥ 0 ;
- porcelets vivants + porcelets morts $\leq$ porcelets nés ;
- la création du lot naissance n'est pas automatique ;
- l'utilisateur doit confirmer la création du lot naissance ;
- le lot naissance doit être lié au lot mère et au groupe de reproduction origine.

8.4 Page analyse reproductive d'un lot femelle

Élément	Détail
Vue Spring MVC	reproduction/analyseLot
Controller	AnalyseReproductionController

Affichage à prévoir

- total femelles ;
- prêtes mais jamais saillies ;
- déjà reproductrices et encore aptes ;
- en cycle ;
- à surveiller ;
- à retirer de la reproduction ;
- taux d'aptitude globale ;
- taux recommandé ;
- taux de fertilité observé ;

- décision automatique ;
- commentaire.

Exemple d'affichage

```

LOT-F-002
Total femelles : 25

Prêtes mais jamais saillies : 12
Déjà reproductrices et encore aptes : 8
En cycle : 0
À surveiller : 3
À retirer de la reproduction : 2

Taux d'aptitude globale : 92 %
Taux recommandé : 80 %
Taux de fertilité observé : 75 %

Décision :
Le lot est encore utilisable pour la reproduction.
20 femelles sont recommandées.
3 femelles doivent être surveillées.
2 femelles ne sont plus recommandées.

```

Fonctions à créer

Élément	Détail
Controller	<code>@GetMapping("/reproduction/analyse/lots/{lotId}") String analyseLot(...)</code>
Controller	<code>@PostMapping("/reproduction/analyse/generer/{lotId}") String genererAnalyse(...)</code>
Service	<code>AnalyseReproductionLotDTO analyserLot(Long lotId)</code>
Service	<code>BigDecimal calculerTauxAptitudeGlobal(...)</code>
Service	<code>BigDecimal calculerTauxRecommande(...)</code>
Service	<code>BigDecimal calculerTauxFertiliteObserve(...)</code>
Service	<code>String genererDecision(AnalyseReproductionLotDTO dto)</code>
Repository	<code>RepartitionReproductiveLotRepository.findByLotId(Long lotId)</code>
Repository	<code>GroupeReproductionRepository.findByLotFemelleId(Long lotId)</code>
Repository	<code>AnalyseReproductionLotRepository.save(...)</code>

Tables utilisées

- lots_porcs ;

- repartitions_reproductives_lots ;
- groupes_reproduction ;
- analyses_reproduction_lots.

Règles et validations

- l'analyse reproductive ne concerne que les lots femelles ;
- le taux de fertilité observé se calcule uniquement sur les femelles déjà saillies ;
- les femelles jamais saillies ne sont pas considérées comme infertiles ;
- les femelles à surveiller peuvent encore être utilisées, mais avec risque ;
- les femelles à retirer ne sont plus recommandées pour la reproduction.

8.5 Page alertes reproduction

Élément	Détail
Vue Spring MVC	<code>reproduction/alertes</code>
Controller	<code>AlerteReproductionController</code>

Affichage à prévoir

- type d'alerte ;
- groupe concerné ;
- lot concerné ;
- message ;
- date alerte ;
- statut ;
- actions lire / traiter.

Fonctions à créer

Élément	Détail
Controller	<code>@GetMapping("/reproduction/alertes") String listAlertes(Model model)</code>
Controller	<code>@PostMapping("/reproduction/alertes/{id}/lire") String marquerCommeLue(...)</code>
Controller	<code>@PostMapping("/reproduction/alertes/{id}/traiter") String traiter(...)</code>
Service	<code>List<AlerteReproduction> listerAlertesActives()</code>
Service	<code>void genererAlertesMiseBasProche()</code>
Service	<code>String marquerCommeLue(Long id)</code>
Service	<code>String traiter(Long id)</code>

Tables utilisées

- alertes_reproduction ;
- groupes_reproduction ;
- lots_porcs.

Règles et validations

- une alerte est générée lorsque la mise bas prévue est proche ;
- une alerte traitée ne doit plus apparaître comme active ;
- une alerte peut être liée à un groupe de reproduction ou à un lot.

9. Module santé

9.1 Vaccins

Élément	Détail
Vue Spring MVC	sante/vaccins
Controller	VaccinController

Fonctionnalités

- liste des vaccins ;
- ajout vaccin ;
- modification vaccin ;
- désactivation simple.

Tables utilisées

- vaccins.

9.2 Vaccinations

Élément	Détail
Vue Spring MVC	sante/vaccinations
Controller	VaccinationController

Fonctionnalités

- enregistrer une vaccination pour un lot ;
- date de vaccination ;
- date de rappel ;
- observation.

Règles

- vaccination liée à un lot existant ;
- date rappel $\geq$ date vaccination ;
- rappel visible dans le dashboard.

Tables utilisées

- vaccinations ;
 - vaccins ;
 - lots_porcs.
-

9.3 Suivis sanitaires

Élément	Détail
Vue Spring MVC	sante/suivis
Controller	SuiviSanitaireController

Fonctionnalités

- diagnostic ;
- maladie ;
- traitement ;
- date traitement ;
- date guérison ;
- observation.

Règles

- suivi sanitaire lié à un lot ;
- date traitement $\geq$ date diagnostic ;
- date guérison $\geq$ date diagnostic ;
- les cas sanitaires en cours doivent apparaître dans le dashboard.

Tables utilisées

- suivis_sanitaires ;
 - maladies ;
 - traitements ;
 - lots_porcs.
-

10. Module commerce

10.1 Clients

Élément	Détail
Vue Spring MVC	<code>commerce/clients</code>
Controller	<code>ClientController</code>

Fonctionnalités

- liste clients ;
- ajout client ;
- modification client ;
- détail client ;
- historique ventes.

Tables utilisées

- clients ;
- ventes.

10.2 Ventes

Élément	Détail
Vue Spring MVC	<code>commerce/ventes</code>
Controller	<code>VenteController</code>

Fonctionnalités

- créer une vente ;
- ajouter des détails de vente ;
- valider une vente ;
- annuler une vente ;
- générer un reçu PDF.

DTO

```
VenteDTO(  
    Long id,  
    Long clientId,  
    LocalDate dateVente,  
    String statut,  
    List<DetailVenteDTO> details  
)
```

```
DetailVenteDTO(  
    Long lotId,
```

```
Integer quantite,  
BigDecimal poidsTotal,  
BigDecimal prixUnitaire,  
BigDecimal montant  
)
```

Règles

- quantité vendue > 0 ;
- quantité vendue <= effectif actuel du lot ;
- montant >= 0 ;
- validation de la vente diminue l'effectif du lot ;
- validation génère un mouvement de type `VENTE` ;
- le reçu PDF remplace la facturation avancée.

Tables utilisées

- ventes ;
- details_vente ;
- clients ;
- lots_porcs ;
- mouvements_lots_porcs.

11. Module finance

11.1 Dépenses

Élément	Détail
Vue Spring MVC	<code>finance/depenses</code>
Controller	<code>DepenseController</code>

Fonctionnalités

- liste dépenses ;
- ajout dépense ;
- catégorie dépense ;
- montant ;
- date dépense ;
- description.

Règles

- montant > 0 ;
- date dépense non future ;
- les dépenses sont utilisées pour calculer le bénéfice net.

Tables utilisées

- dépenses ;
- categories_depenses.

12. Module stocks alimentaires

12.1 Ingrédients

Élément	Détail
Vue Spring MVC	stocks/ingredients
Controller	IngredientController

Fonctionnalités

- liste ingrédients ;
- ajout ingrédient ;
- stock actuel ;
- seuil alerte ;
- unité.

Règles

- stock actuel ≥ 0 ;
- seuil alerte ≥ 0 ;
- stock faible visible sur dashboard.

Tables utilisées

- ingredients.

12.2 Mouvements de stock

Élément	Détail
Vue Spring MVC	stocks/mouvements
Controller	MouvementStockController

Fonctionnalités

- entrée stock ;
- sortie stock ;
- ajustement stock ;
- historique mouvements.

Règles

- quantité > 0 ;
- sortie impossible si stock insuffisant ;
- entrée augmente le stock ;
- sortie diminue le stock.

Tables utilisées

- mouvements_stock_aliment ;
- ingredients.

13. Module ressources humaines simplifié

13.1 Employés

Élément	Détail
Vue Spring MVC	rh/employes
Controller	EmployeController

Fonctionnalités

- liste employés ;
- ajout employé ;
- modification employé ;
- poste ;
- téléphone ;
- date embauche.

Tables utilisées

- employes.

13.2 Présences

Élément	Détail
Vue Spring MVC	rh/presences
Controller	PresenceController

Fonctionnalités

- pointer présence ;
- présent ;

- absent ;
- retard ;
- historique simple.

Règles

- une seule présence par employé et par date ;
- statut obligatoire.

Tables utilisées

- presences ;
- employes.

14. Module rapports

14.1 Rapports disponibles

Rapport	Format
Rapport sanitaire	PDF
Rapport commercial	PDF
Rapport financier	PDF
Rapport reproduction	PDF
Export ventes	Excel
Export lots	Excel
Export reproduction	Excel
Export stocks	Excel

Controller

RapportController

Routes

- /rapports
- /rapports/sanitaire/pdf
- /rapports/commercial/pdf
- /rapports/financier/pdf
- /rapports/reproduction/pdf
- /rapports/export/excel

Règles

- les rapports sont générés depuis les données existantes ;
- il n'y a pas de table `rapports_generes` obligatoire ;
- seules les données validées sont exportées ;
- les exports PDF et Excel sont obligatoires pour la validation du projet.

15. Module import / export

15.1 Import Excel

Élément	Détail
Vue Spring MVC	<code>imports/form</code>
Controller	<code>ImportExportController</code>

Imports prévus

- lots ;
- clients ;
- vaccinations ;
- ingrédients.

Règles

- fichier Excel obligatoire ;
- colonnes obligatoires vérifiées ;
- lignes invalides rejetées avec message ;
- import tracé dans `imports_exports` .

15.2 Export Excel

Exports prévus

- lots ;
- ventes ;
- stocks ;
- analyse reproduction ;
- groupes de reproduction.

Règles

- export au format Excel ;
- données filtrables ;
- opération tracée dans `imports_exports` .

15.3 Export PDF

Exports PDF prévus

- reçu de vente ;
- rapport sanitaire ;
- rapport financier ;
- rapport reproduction ;
- fiche détail lot.

16. Base de données finale

Sécurité

- rôles ;
- utilisateurs.

Références métier

- races ;
- parametres_reproduction_race.

Cheptel

- lots_porcs ;
- mouvements_lots_porcs ;
- pesees_lots.

Reproduction par groupes

- statuts_reproductifs ;
- repartitions_reproductives_lots ;
- groupes_reproduction ;
- analyses_reproduction_lots ;
- alertes_reproduction.

Santé

- vaccins ;
- vaccinations ;
- maladies ;
- traitements ;
- suivis_sanitaires.

Commerce

- clients ;
- ventes ;
- details_vente.

Finance

- categories_depenses ;
- depenses.

Stocks

- ingredients ;
- mouvements_stock_aliment.

Ressources humaines

- employes ;
- presences.

Import / Export

- imports_exports.
-

17. Tables supprimées

Les tables suivantes ne doivent plus être utilisées :

- reproducteurs ;
 - cycles_production ;
 - evenements_reproduction ;
 - type_evenements_reproduction ;
 - paiements ;
 - factures ;
 - melanges ;
 - melange_ingredients ;
 - distributions_aliment ;
 - salaires_employes ;
 - permissions ;
 - role_permissions.
-

18. Tables principales détaillées

18.1 lots_porcs

Colonnes principales :

- id ;
- code_lot ;
- date_creation ;
- race_id ;
- sexe ;
- objectif ;
- origine ;
- effectif_initial ;
- effectif_actuel ;
- statut ;
- lot_parent_id ;
- groupe_reproduction_origine_id ;
- description.

Règles :

- sexe = MALE ou FEMELLE ;
 - pas de lot mixte ;
 - effectif initial > 0 ;
 - effectif actuel >= 0.
-

18.2 groupes_reproduction

Colonnes principales :

- id ;
- code_groupe ;
- lot_femelle_id ;
- lot_male_id ;
- nombre_femelles_concernees ;
- nombre_males_utilises ;
- date_saillie ;
- duree_gestation_jours ;
- date_prevue_mise_bas ;
- date_mise_bas_reelle ;
- nb_femelles_gestantes ;
- nb_femelles_non_gestantes ;
- nb_femelles_mise_bas ;
- nb_porcelets_nes ;
- nb_porcelets_vivants ;
- nb_porcelets_morts ;
- statut ;

- observation.

Règles :

- lot_femelle_id obligatoire ;
- nombre femelles concernées > 0 ;
- date prévue de mise bas calculée ;
- vivants + morts <= nés ;
- le groupe peut être clôturé après mise bas.

18.3 repartitions_reproductives_lots

Colonnes principales :

- id ;
- lot_id ;
- statut_reproductif ;
- quantite ;
- date_mise_a_jour.

Règles :

- quantité >= 0 ;
- la somme des répartitions ne doit pas dépasser l'effectif actuel du lot femelle ;
- utilisé pour afficher l'état reproductif global du lot.

18.4 analyses_reproduction_lots

Colonnes principales :

- id ;
- lot_id ;
- date_analyse ;
- nb_pretees_jamais_saillies ;
- nb_deja_reproductrices_aptas ;
- nb_en_cycle ;
- nb_a_surveiller ;
- nb_a_retirer_reproduction ;
- nb_femelles_total ;
- nb_femelles_saillies_total ;
- nb_femelles_gestantes_total ;
- taux_apptitude_global ;
- taux_recommande ;
- taux_fertilite_observe ;
- decision ;
- commentaire.

Formules :

```
taux_apptitude_global =  
(prêtes + déjà aptes + à surveiller) / total femelles × 100
```

```
taux_recommande =  
(prêtes + déjà aptes) / total femelles × 100
```

```
taux_fertilite_observe =  
femelles gestantes / femelles saillies × 100
```

18.5 alertes_reproduction

Colonnes principales :

- id ;
- groupe_reproduction_id ;
- lot_id ;
- type_alerte ;
- message ;
- date_alerte ;
- statut.

Types d'alerte :

- mise bas proche ;
- retard mise bas ;
- surveillance lot ;
- réforme recommandée.

19. Cas d'utilisation principal

Situation de départ

L'utilisateur crée deux lots :

```
LOT-F-001  
Sexe : FEMELLE  
Objectif : REPRODUCTION  
Effectif : 4
```

```
LOT-M-001  
Sexe : MALE
```

Objectif : REPRODUCTION
Effectif : 2

Le système affiche :

LOT-F-001 | Femelle | Reproduction | 4
LOT-M-001 | Mâle | Reproduction | 2

Analyse initiale

LOT-F-001
Total femelles : 4

Prêtes mais jamais saillies : 4
Déjà reproductrices et encore aptes : 0
En cycle : 0
À surveiller : 0
À retirer de la reproduction : 0

Aptitude reproductive : 100 %
Fertilité observée : non calculée

Création d'un groupe reproductif

L'utilisateur décide de saillir seulement 2 femelles.

Il saisit :

Lot femelle : LOT-F-001
Nombre femelles concernées : 2
Lot mâle utilisé : LOT-M-001
Nombre mâles utilisés : 1
Date saillie : 01/07/2026
Durée gestation : 114 jours

Le système crée :

GR-001
Femelles concernées : 2
Date prévue mise bas : 23/10/2026
Alerte mise bas : 18/10/2026

Le lot devient :

LOT-F-001
Total femelles : 4
Femelles en cycle : 2
Femelles disponibles : 2

Alerte

Cinq jours avant la mise bas :

Alerte reproduction

Le groupe GR-001 approche de la mise bas.
Date prévue : 23/10/2026
Action recommandée :
Surveiller le groupe concerné et préparer l'espace de mise bas.

Confirmation de mise bas

L'utilisateur saisit :

Date réelle : 23/10/2026
Porcelets nés : 18
Porcelets vivants : 16
Porcelets morts : 2

Le système propose :

Créer un nouveau lot naissance ?

Après confirmation :

LOT-N-001
Origine : NAISSANCE
Lot mère : LOT-F-001
Groupe origine : GR-001
Effectif initial : 16
Effectif actuel : 16

Résultat final

LOTS ACTIFS

LOT-F-001 | Femelle | Reproduction | 4

LOT-M-001 | Mâle | Reproduction | 2
LOT-N-001 | Naissance | 16

Dashboard :

Groupes actifs : 0
Mises bas confirmées : 1
Porcelets vivants : 16
Fertilité observée : 100 %

20. Critères de validation fonctionnelle

Le projet est considéré comme fonctionnel si :

- un lot peut être créé ;
- un lot ne peut pas être mixte ;
- un lot femelle peut avoir une répartition reproductive ;
- une partie seulement d'un lot femelle peut entrer en reproduction ;
- un groupe de reproduction peut être créé ;
- la date prévue de mise bas est calculée automatiquement ;
- une alerte est générée avant la mise bas ;
- une mise bas peut être confirmée ;
- un lot naissance peut être créé après confirmation ;
- l'analyse reproductive du lot est affichée ;
- le taux de fertilité observé est calculé correctement ;
- le taux d'aptitude globale est calculé correctement ;
- les ventes diminuent l'effectif du lot ;
- les mouvements expliquent les variations d'effectif ;
- les exports PDF fonctionnent ;
- les exports Excel fonctionnent ;
- les imports Excel fonctionnent.

21. Conclusion technique

Cette version de MADAPORC est centrée sur un vrai cœur métier :

La gestion reproductive par groupes et la traçabilité complète des lots porcins.

Le système permet de suivre un lot femelle, de sélectionner seulement une partie du lot pour la reproduction, de calculer la date prévue de mise bas, de générer une alerte, de confirmer la mise bas et de créer un nouveau lot naissance.

Cette approche évite le simple CRUD et donne au projet une vraie valeur métier adaptée à l'élevage porcin.